2章

業務知識を発見する

本番環境にリリースされるのは開発者の理解（あるいは誤解）です。業務エキスパートの知識ではありません。

——アルベルト・ブランドリーニ＊1

＊1　［訳注］第12章で説明するイベントストーミングの考案者。

前の章で、事業活動を探求するところから出発しました。事業領域はどこか、そして競合する他社と差別化する基本方針は何かを特定する方法を学びました。つまり、業務領域の細分化とカテゴリー（中核、補完、一般）です。

この章も事業活動の分析を続けます。しかし、方向は「深さ」に変わります。事業活動を深く理解するために、それぞれの業務領域の内部、つまり業務機能と業務ロジックに焦点を合わせます。この章で学ぶのは、意図を適切に伝え、知識を共有するためのドメイン駆動設計の基本的な道具である同じ言葉（Ubiquitous Language）です。この章では、事業活動の複雑な細部を知る方法として同じ言葉を使います。以降の章では、同じ言葉を業務ロジックのモデリングと実装の方法として使います。

2.1　事業の課題

私たちがソフトウェアを開発するのは事業課題を解決するためです。ここでいう「課題」は数学の問題やなぞなぞとは違います。あらかじめ決まった答えはありません。事業活動における課題とは、もっと広い意味を持ちます。業務プロセスやワークフローの最適化、手作業の最小化、経営資源（ヒト、モノ、カネ）の管理、意思決定の支援、そして業務データの管理などです。

事業課題は、事業レベルと業務領域レベルの両方に現れます。事業の目標は、顧客がかかえる課題の解決です。第1章のクロネコヤマトの例で考えてみましょう。顧客の課題は荷物の迅速な移動です。クロネコヤマトは、配送のやり方を最適化して、顧客の課題を解決します。

業務領域レベルでは、もっと具体的な課題の解決に取り組みます。たとえば知識管理（ナレッジマネジメント）の領域では、情報の保存と利用の方法を最適化します。決済業務であれば請求や回収のやり方を最適化します。会計業務であれば会社の資金を追跡します。

2.2　知識の発見

役に立つソフトウェアを設計しようとすれば、事業活動について、少なくとも、基本レベルの知識を把握することが必要です。第1章で検討したように、この知識を持っているのは業務エキスパート、つまり業務の当事者として複雑な業務に日々取り組んでいる人たちです。ソフトウェア開発者が業務エキスパートになる必要はありません。なろうとしてもなれないでしょう。ソフトウェア開発者がやるべきことは、業務エキスパートの考え方を理解し、業務エキスパートが使っている用語を自分たちも使うことです。

効果的であるためには、ソフトウェアは業務エキスパートが業務課題をどのようにとらえているか、つまり業務エキスパートのメンタルモデルをうまく表現することが必要です。事業課題と要求事項の背景を理解しないまま開発したソフトウェアは、要求事項をソースコードに「翻訳」したにすぎません。きわめて重要な特殊ケースが要求事項から漏れていたらどうなるでしょうか。あるいは事業の基本的な概念の表現に失敗し、将来の要求に対応できないモデルを実装してしまえば、どうなるでしょうか。

アルベルト・ブランドリーニが言っているとおりです＊2。ソフトウェア開発は学びのプロセスです。動くコードは学びの副産物です。ソフトウェア開発プロジェクトが成功するかどうかは、業務エキスパートとソフトウェア技術者が知識を効果的に共有できるかどうかで決まります。よいソフトウェアを作るためには、解決すべき課題の理解が必要です。

＊2　Alberto Brandolini著『Introducing EventStorming』（https://www.eventstorming.com/book）（Leanpub、出版年不明）

知識を効果的に共有するためには、業務エキスパートとソフトウェア技術者が効果的に意図を伝えあうことが必要です。まず、ソフトウェア開発プロジェクトでありがちな、意図の伝達がうまくいっていない状況を見てみましょう。

2.3　意図の伝達

さまざまな役割の関係者が協力することは、ほぼすべてのソフトウェア開発プロジェクトで必要でしょう。つまり、業務エキスパート、プロダクト責任者、ソフトウェア技術者、UI/UXデザイナー、プロジェクト管理者、テスト担当者、分析担当者など、さまざまな関係者の協力です。協力のやり方はいろいろありますが、何を生み出せるかは、どのように「一緒に働く」かに依存します。たとえば、解決すべき課題が何であるかをすべての関係者が同意できているでしょうか。あるいは、機能要件や非機能要件の想定が関係者によって異なっていないでしょうか。ソフトウェア開発を成功させるには、ソフトウェア開発に関係するあらゆる事項について、関係者の間で合意し整合させることが不可欠です。

ソフトウェア開発プロジェクトが失敗した原因を調査した報告書によれば、効果的な意図の伝達は、知識の共有とプロジェクトの成功に不可欠です**＊3**。

＊3　Goparaju Purna Sudhakar, “A Model of Critical Success Factors for Software Projects（ソフトウェア開発プロジェクトの成功要因のモデル）”, Journal of Enterprise Information Mangement, 25*(6), 537-538, (2012). 参照：
https://www.emerald.com/insight/content/doi/10.1108/17410391211272829/full/htm

さらに言えば、意図の伝達はきわめて重要であるにもかかわらず、それがうまくできているプロジェクトはほとんどありません。業務側とソフトウェア開発者とが直接対話しないプロジェクトはたくさんあります。業務知識は業務エキスパートからソフトウェア開発者への一方向で渡されます。両者の中間に「翻訳」を担当する人たちが介在します。要件定義の担当者、プロダクト責任者、そしてプロジェクト管理者です。ソフトウェア開発でよく見かける知識の伝達は**図2‐1**のようになります。

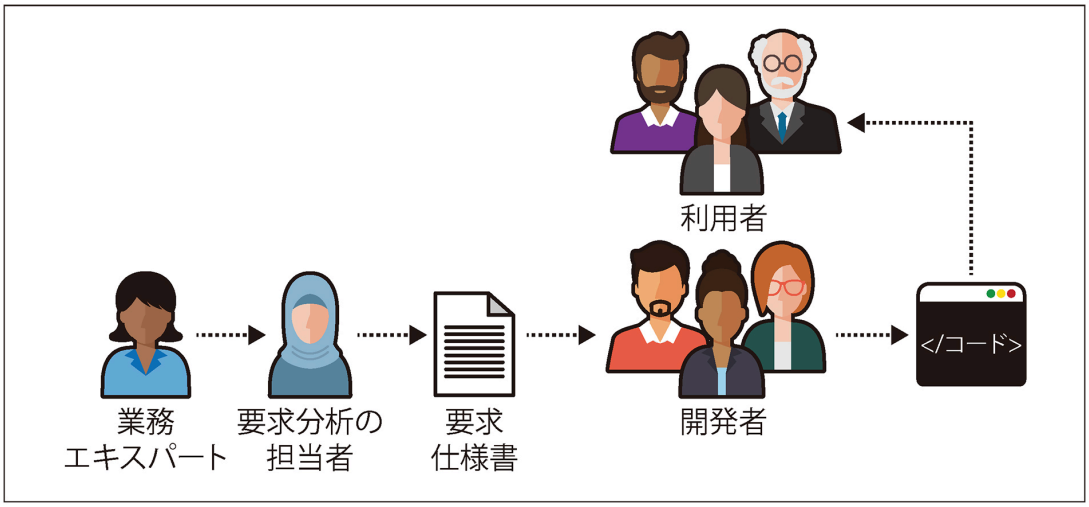

図2-1　ソフトウェア開発における典型的な知識の共有

ソフトウェア開発の伝統的なやり方では、業務知識を開発者にわかりやすい形式、つまり分析モデルに変換します。分析モデルはソフトウェアへの要求仕様の記述です。要求仕様の背景にある事業活動を理解するためのものではありません。開発者向けの形式に変換することは悪いことではないでしょう。しかし、そういった変換は知識の共有には有害です。変換により情報が欠落します。分析モデルで言えば、事業課題の解決に不可欠な業務知識がソフトウェア開発者に届きません。情報が欠落する変換は分析モデルだけではありません。分析モデルから基本設計書への変換、基本設計書から詳細設計書への変換、そして詳細設計書からソースコードへの変換が行われます。そしてほとんどのドキュメントは更新されず、あっという間に役に立たなくなります。保守担当者に伝わる業務知識はソースコードだけになります。業務知識がコードに実装されるまでの変換の流れは図2‐2のようになります。

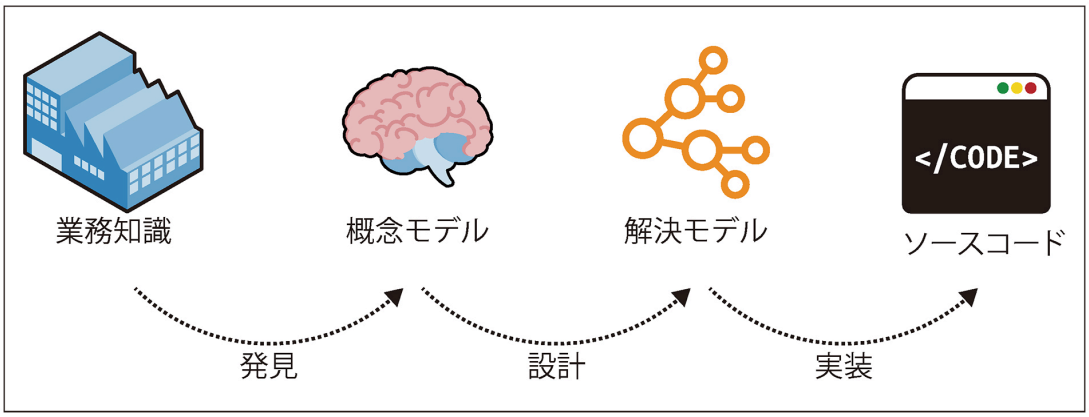

図2-2　モデルの変換

こういうソフトウェア開発のやり方は伝言ゲームです。つまり、業務知識は伝言ゲームの途中で誤って伝わります。そういう誤った情報をもとにして開発

すれば誤ったソフトウェアが生まれます。あるいは本来の課題とはまったく別の課題を解決するソフトウェアが生まれます。どちらにしても結果は一つです。つまり、ソフトウェア開発プロジェクトの失敗です。

ドメイン駆動設計は、業務エキスパートからソフトウェア技術者に業務知識を伝えるもっとうまい方法を提案します。それが同じ言葉です。

2.4 「同じ言葉」とは何か？

同じ言葉はドメイン駆動設計の土台となる手法です。考え方は単純です。つまり、お互いの意図を効率的に伝えたいなら、変換するのではなく、同じ言葉を使わなければいけません。

この考え方は常識のように思えます。しかし、ヴォルテール（18世紀のフランスの思想家）が言ったように、常識が普通とは限りません。伝統的なソフトウェア開発のやり方は次のような変換を前提としています。

- 業務知識　⇒　分析モデル
- 分析モデル　⇒　要求仕様書
- 要求仕様書　⇒　システム設計書
- システム設計書　⇒　ソースコード

ドメイン駆動設計は、業務知識の変換を繰り返すかわりに、事業活動を表現するための単一の言語を育成しようとします。それが同じ言葉です。

ソフトウェアが対象としている業務を表現する時は、プロジェクトのすべての関係者（ソフトウェア技術者、プロダクト責任者、業務エキスパート、UI/UXデザイナー）は、必ず同じ言葉を使わなければいけません。業務エキスパートが業務の内容を説明しやすいことがもっとも重要です。同じ言葉とは、事業活動を表現すると同時に、業務エキスパートの発想や考え方をそのまま表現できる言葉です。

同じ言葉とその用語を継続的に使うことによってのみ、すべての関係者の間で共通理解を生み出すことが可能です。

2.5　業務で使う言葉

同じ言葉は業務で使う言葉です。この点はきわめて重要です。同じ言葉に含まれるのは業務用語だけです。技術用語を含めてはいけません。シングルトン

やファクトリーパターンが何であるかを業務エキスパートに教えようとしないでください。同じ言葉の目的は、業務エキスパートの頭の中にある事業活動についての知識や考え方を、わかりやすく整理することです。

2.5.1　同じ言葉の使い方

広告キャンペーンの管理システムを考えてみましょう。次の表現はどうでしょうか。

- 広告キャンペーンはいろいろな広告素材を使って表示できる
- キャンペーンの配信には少なくとも一つのアクティブな配信先が必要である
- 販売報奨金はコンバージョンが承認された後で発生する

いずれもオンライン広告業界の用語を使った説明です。つまり広告キャンペーンの業務エキスパートの見方を表現しています。

一方、以下の表現はきわめて技術的な説明です。つまり同じ言葉の考え方とは異なる説明のしかたです。

- HTML形式のiframeを使って広告を表示する
- 配信先テーブルに少なくとも一つの有効なレコードがある時のみキャンペーンを配信できる
- 販売報奨金は配信済テーブルと承認済テーブルのレコードを関連づけて算出する

こういう技術的な説明は業務エキスパートには何のことかわかりません。ソフトウェア開発者が事業領域を技術的な観点だけで見ているとどうなるでしょうか。そういう開発者は業務ロジックやその機能が必要な理由を正しく理解できません。つまり事業の役に立つソフトウェアを生み出すことができません。

2.5.2　一貫性

同じ言葉は正確で一貫していることが必要です。言葉の意味をあれこれ推測するようではいけません。業務ロジックを明確に表現できることが必要です。

あいまいな言葉は意図が伝わりません。ですから、同じ言葉を構成するそれぞれの用語の意味は一つでなければいけません。意味が伝わりにくい言葉の例と伝わりにくさをどうやって改善するかを見てみましょう。

2.5.2.1　あいまいな用語

たとえばポリシー（policy）という用語は、ある事業分野では複数の意味を持ちます。「統制ルール」と「保険契約」です。人間どうしの会話であれば、話の前後関係からどちらの意味か判断できます。しかしソフトウェアはそういうあいまいさをうまく扱えません。ポリシーをそのままモデルの要素とし、Policyクラスとして実装するのは混乱の元です。

同じ言葉は、一つ用語が一つの意味に対応することが必要です。ですから、ポリシー（policy）としてモデルを表現するのではなく、統制ルール（regulatory rule）と保険契約（insurance contract）という二つの用語を使ってモデルを明確に表現すべきです。

2.5.2.2　同義語

同じ言葉では、二つの用語を同じ意味で使うことはできません。たとえば、多くのシステムは「利用者」という用語を使います。しかし、業務エキスパートの言葉の使い方を注意深く観察すると、「利用者」と同じ意味で使われるさまざまな用語を見つけられるでしょう。たとえば、「利用者」「サイト訪問者」「管理者」「会員」などです。

こういう同義語（意味が同じ別の言葉）は最初はたいした問題には見えないでしょう。しかし、ほとんどの場合、言葉が違えば何か異なる意図があるはずです。さきほどの「サイト訪問者」と「会員」は技術的にはどちらもシステムのユーザーです。しかし、ほとんどのシステムでは未登録ユーザーと登録済ユーザーは異なる種類の利用者を表現し、その行動も異なります。たとえば「サイト訪問者」に関するデータは主に分析目的で使います。それに対し「会員」は実際にシステムとその機能を利用します。

意図が違うのであれば、それぞれの用語を特定の文脈ごとに使い分けましょう。使われる用語の意味の違いを理解すれば、モデルは単純で明確になり、さまざまなビジネス要素（エンティティ）の実装も単純で明確になります。

2.6　事業活動のモデル

同じ言葉を別の観点、つまりモデリングの観点から眺めてみましょう。

2.6.1　モデルとは何か

モデルとは、現実のモノや出来事を簡略化した表現です。特定の側面を意図的に強調し、一方で、それ以外の側面を意図的に除外します。モデルは用途を限定した抽象化です。

——レベッカ・ワーフスブラック

モデルは現実世界のコピーではありません。現実世界の仕組みがどうなっているかを理解するための作り物です。

モデルのわかりやすい例として地図があります。道路地図、地形図、世界地図、地下鉄の路線図など、どれもモデルです（図2‐3）。

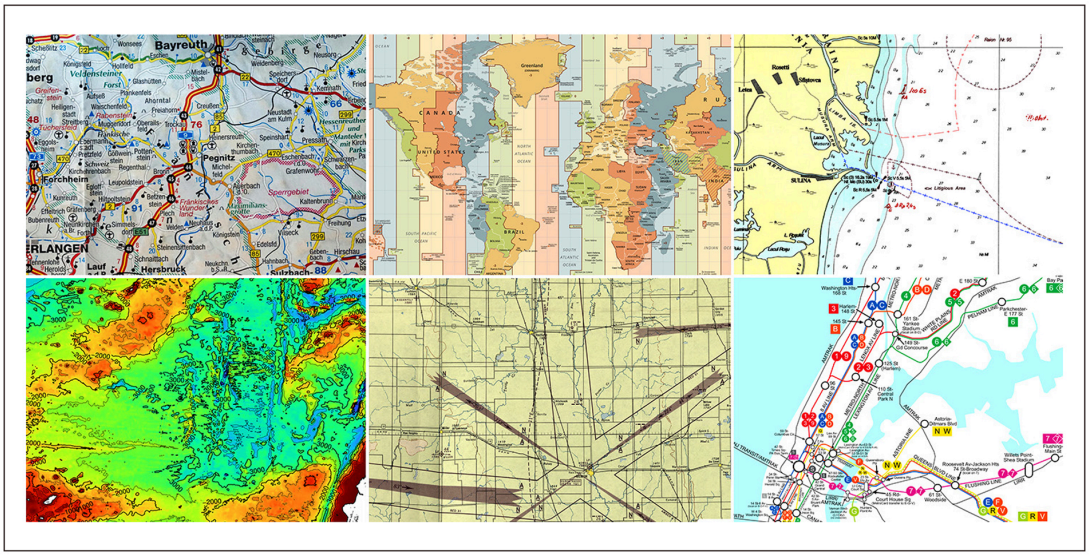

図2-3　それぞれの地図は、現実世界のさまざまな側面を表現している。道路、タイムゾーン、航路、地形、航空路、地下鉄の路線など

どの地図も現実世界のすべての詳細を表現しているわけではありません。そのかわり、それぞれの地図は特定の用途、つまり解決しようとしている課題をサポートするために十分なデータだけを含みます。

2.6.2　効果的なモデル

モデルには必ず目的があります。役に立つモデルはその目的にそった内容だけを表現します。たとえば世界地図に地下鉄の駅は不要です。一方で地下鉄の路線図からは正確な距離を知ることはできません。どの地図もその目的に合致する情報だけを提供します。

この点はきわめて重要です。つまり、役に立つモデルは現実世界のコピーではありません。そのかわり、モデルは特定の課題を解決することを目指し、そ

の目的にそった情報だけを提供しなければなりません。統計学者のジョージ・ボックスが言っています。「あらゆるモデルはまちがっている。しかし役に立つモデルはある」。

モデルは抽象化です。抽象化とは、複雑さをうまく扱うために、課題解決には必要としない詳細を取り除き、課題解決に必要な部分を抜き出す活動です。それとは反対に、必要な部分まで取り除いてしまった抽象化や、ノイズにしかならない不要な情報を残したままの抽象化は役に立ちません。エドガー・ダイクストラが論文「謙虚なプログラマー」＊4で書いているとおりです。抽象化の目的は、対象をあいまいに表現することではありません。抽象化とは、対象を「きわめて正確」に表現するための、新たなとらえ方を生み出す活動です。

＊4　Edsger W. Dijkstra, “The Humble Programmer”(https://oreil.ly/LXd4W).

2.6.3　事業活動のモデルを作る

同じ言葉を育てれば、事業活動のモデルも成長します。事業活動のモデルは、業務の目的や内容を業務エキスパートはこうとらえているだろうと推測したものです。事業活動のモデルは、事業活動に登場するヒトやモノ、事業活動で発生する出来事、原因と結果の関係、そして守るべき決めごとを表現することが必要です。

事業活動のありとあらゆる詳細を同じ言葉を使って表現しようというのではありません。それは、あらゆる関係者を業務エキスパートにしようとするのと同じです。そうではなく、目的とするシステムの実装を可能にするために、事業活動の特定の側面だけを含んだモデルを作る、ということです。つまり、ソフトウェアで解決しようとしている特定の課題に取り組むためのモデルです。第3章以降では、同じ言葉を使うことで、ソフトウェア設計と実装をどのように推進できるかを説明します。

開発チームと業務エキスパートがお互いの意図を効果的に伝えることはきわめて重要です。事業活動が複雑になるほど、効果的な意図の伝達が重要になります。事業活動が複雑であれば、業務ロジックのモデルを作りコードで表現することは難しくなります。複雑な業務内容やその背景にある基本原則についてのちょっとした誤解が、ソフトウェアの深刻な不具合につながります。業務を適切に理解できているかを確かめるための、信頼できるたった一つの方法は、

業務エキスパートが理解できる言葉、つまり業務の言葉を使って業務エキスパートと会話することです。

2.6.4　継続的に取り組む

同じ言葉を形作るには、その言葉を使っている業務エキスパートとの会話が必要です。業務エキスパートと会話することで、不正確な表現、まちがった解釈、あるいは事業活動についての誤解を検出できます。

ソフトウェア開発の利害関係者は誰でも、プロジェクトに関わるあらゆる意図の伝達に、一貫して同じ言葉を使わなければなりません。そうすることで、業務知識が広がり、事業活動についての共通理解が育ちます。同じ言葉をプロジェクトの隅々まで行き渡らせることが必要です。つまり、要件定義、テスト、各種ドキュメント、そしてソースコードさえ同じ言葉を使うことが必要です。

もっとも重要なことは、同じ言葉を時間とともに成長させることです。言葉の妥当性を常に検証し進化させていくことが必要です。同じ言葉を毎日使い続けることで、事業活動の本質を深く理解できるようになります。そういう事業理解のブレークスルーが起きた場合、開発者が獲得した新たな業務知識を使って、同じ言葉を進化させなければいけません。

2.6.5　道具の利用

同じ言葉の発見と整理に役に立つ道具を紹介します。

たとえばwikiは同じ言葉を発見し文書化する「用語集」として使えます。そういう用語集があればチームに新たなメンバーが参加しやすくなります。用語集は新たなメンバーが事業活動を学ぶ時にまっさきに参照すべき情報源になります。

この用語集は全員で更新することがたいせつです。同じ言葉に変化があれば、それに気がついたメンバーが率先して用語集を更新します。チームリーダーやアーキテクトだけが用語集を管理する中央集権的なやり方とは真逆のやり方です。

常に更新されている用語集が役に立つことは明らかです。しかし用語集には本質的な限界があります。用語集は「名詞」、つまり業務で扱うモノ、プロセス、役割などの名前の整理には役に立ちます。名詞はもちろん重要です。しか

し、本当に重要なのは業務活動の振る舞いをとらえることです。振る舞いは名詞と関連づけられた動詞の単なるリストではありません。振る舞いの実体は業務ロジックです。つまり、業務活動を統制する業務ルール、前提条件そして制約です。こういう概念は、用語集の形式ではうまく表現できません。したがって、用語集が役に立つのは、振る舞いの表現に適した他の手法と組み合わせた場合です。たとえば、ユースケースやGherkinテストです。GherkinはBDD（振る舞い駆動開発）で使われるテスト記法の一つです。

Gherkin記法＊5で記述した自動テストは、同じ言葉を文書化するだけでなく、業務エキスパートとソフトウェア技術者との距離を縮めます。同じ言葉を使って記述したテストコードを業務エキスパートが読むことで妥当性を検証できるでしょう＊6。Gherkin記法で書いたテストコードを見てみましょう。

```
Senerio: 新規案件をサポート担当者に通知する
  Given 山田太郎さんが以下の新規案件を追加する:
  """
  AWSのInfinidashサービスのセットアップについて教えてください
  """
  When 案件チケットを吉田さんに割り当てる
  Then 吉田さんにチケット割り当て通知が届く
```

＊5　https://oreil.ly/WJw3C

＊6　「読める」だけです。業務エキスパートがGherkin記法でテストを「書ける」という幻想は捨てましょう。

このようなテスト記述を体系的に用意することは簡単ではないでしょう。特にプロジェクトの前半ではかなりたいへんです。しかし複雑な業務を対象にする時はこのようなテストが本当に役に立ちます。

同じ言葉の用語がソースコードに適切に使われているかを検査できるツールもあります。たとえばNDepend＊7というツールです。

＊7　［訳注］C#で書かれたソースコードを解析して、記述の一貫性の問題や依存関係の複雑さを視覚的にレポートするツール。

ここで紹介したツールは同じ言葉を使うための補助手段にすぎません。たいせつなことは、同じ言葉を毎日の会話の中で実際に使うことです。ツールを使えば、同じ言葉の文書化を進めることはできます。しかし文書化は、同じ言葉を使った会話のかわりにはなりません。アジャイルソフトウェア開発宣言**＊8**にあるように「プロセスやツールよりも個人と対話」をたいせつにしましょう。

＊8　https://agilemanifesto.org/iso/ja/manifesto.html

2.6.6　困難に立ち向かう

同じ言葉を育てることは、理屈の上では、単純でわかりやすい取り組みです。しかし現実はそうではありません。業務知識を習得するただ一つの確実な方法は、業務エキスパートとの会話です。多くの場合、本当に重要な業務知識は言語化されていません。重要な知識は文書化や体系化がされず、業務エキスパートの頭の中だけに存在しています。言語化されていない知識にたどりつくただ一つの方法は、質問することです。

業務エキスパートにいろいろ質問をするようになると、質問をすることは、すでに存在する知識の発見活動だけではなく、業務エキスパートとモデルを共創する活動であることに気づくでしょう。業務エキスパートであっても、理解があいまいなところやわからないところもあるはずです。たとえば、正常系の業務シナリオはしっかり把握できていても、前提の見直しにつながる特殊なケースを考慮できていないかもしれません。さらに言えば、明確に定義されていない業務の概念にでくわすこともあるでしょう。事業活動の特質について質問することで、そういうあいまいなところや見落としていたことが、いろいろと明らかになるでしょう。特に、中核の業務領域では、そういうことが普通に起きます。この場合、学習のプロセスは相互作用です。つまり、技術者が質問することで、業務エキスパートがその領域をさらに深く理解することを助けます。

ドメイン駆動設計の考え方とやり方を既存システムのプロジェクトに取り入れようとすると、業務内容を表現するある種の言語がすでに存在していることに気がつくでしょう。関係者はその言語を使っています。しかし、ドメイン駆動設計のやり方を意識せずに生み出されたそういう言語は、業務知識を適切に表現できていないかもしれません。たとえば技術的な視点で命名されたデータベースのテーブル名をそのまま使っているかもしれません。関係者の間で言葉の使い方を変えることは簡単ではありません。そういう状況での基本手段は忍

耐です。自分たちがコントロールしやすい場所から手をつけることが必要です。つまり、技術文書やソースコードに業務知識を反映します。

最後に、同じ言葉についてよくある質問です。 技術カンファレンスの参加者から「英語圏ではない国では同じ言葉はその国の言葉を使うべきでしょうか」という質問を受けることがあります。私は、少なくとも、事業活動で関心の対象となるエンティティ名は、英語の名詞を使うことをすすめています。そのほうがソースコードに同じ単語を使いやすいでしょう。

> 日本では、カスタマー、プロダクト、オーダーなどのカタカナ用語であれば、対応する英単語をソースコードに使うという方法があります。しかし、業務エキスパートが、カタカナ言葉ではなく、「顧客」「商品」「注文」などの用語を使っている場合には、本来は、同じ言葉はそれに合わせることになります。その場合、次善の策として、クラス名は英単語を使い、ドキュメンテーションコメントで日本語名を併記して対応づける、などが考えられます。

2.7　まとめ

ソフトウェアプロジェクトを成功させるには、効果的な意図の伝達と知識の共有がきわめて重要です。ソフトウェア技術者がソフトウェアを設計し組み立てるためには、事業活動の理解が不可欠です。

ドメイン駆動設計の同じ言葉は、業務エキスパートとソフトウェア技術者の間にある知識のギャップを埋める効果的な手法です。プロジェクト全体を通じて、いつでもどこでも誰もが同じ言葉を使うことで、意図の伝達と知識の共有が進みます。つまり、会話、文書、テスト、図、そしてソースコードなど、あらゆるところで同じ言葉を使ってください。

意図の伝達をうまくやるためには、同じ言葉から、あいまいさや暗黙の前提を取り除くことが必要です。同じ言葉は一貫性が必要です。あいまいな言葉や同義語を使ってはいけません。

同じ言葉を育てることは継続的な活動です。プロジェクトが進むにつれ、さまざまな業務の知識が見つかります。新たに発見した業務知識を同じ言葉に取り入れ続けることがたいせつです。

wikiを使った用語集やGherkin記法で記述したテストコードは、同じ言葉の文書化と整理に役に立ちます。しかし、同じ言葉を活用するために本当に必要なのは、その言葉を実際に使うことです。つまり、プロジェクトの関係者は、意図を伝えるあらゆる機会に、この言葉を一貫して使わなければいけません。

2.8　演習問題

1. 同じ言葉の定義に貢献できるのは誰ですか？
   A. 業務エキスパート
   B. ソフトウェア技術者
   C. エンドユーザー
   D. プロジェクトのすべての関係者
2. 同じ言葉をどこで使いますか？
   A. 会話の中
   B. 文書の中
   C. ソースコード
   D. 上記のすべて
3. 「この本について」で紹介したウルフデスク社の説明で、どれが業務の用語でしょうか？
4. あなたが現在取り組んでいる、あるいはかつて経験してきたソフトウェアプロジェクトについて、以下を考えてみてください。
   A. あなたが業務エキスパートと会話する時に使える業務の概念をあげてみてください。
   B. 業務に使われる一貫性に欠ける用語、つまり、複数の意味で使われる用語、あるいは同じ意味を指す複数の用語の例をあげてください。
   C. 意図の伝達がうまくいかないことでソフトウェア開発が非効率だった経験がありますか？
5. 異なる部門から参加している業務エキスパートが「ポリシー」という言葉を異なる意味で使っている状況を想像してください。

「ポリシー」という言葉は業務エキスパートの考え方にもとづいています。しかし「一つの言葉には一つの意味」という同じ言葉の基準を満たしていないことに気がついた時、あなたはどう対処しますか？

次の章に進む前に、この難題への取り組み方を考えてみてください。